AI Secure Code Intelligence Platform

Deep Architectural Analysis, Motivation, Utility & Execution Guide

---

1. Motivation: Why Was This Platform Developed?

Modern software engineering relies heavily on rapid iteration, third-party libraries, and AI-assisted code generation tools. However, this speed often comes at the expense of software security. Developers frequently push code containing high-severity security flaws such as **SQL Injection (SQLi)**, **Command Injection**, **Insecure Deserialization**, and **Hardcoded Secret Keys** into production environments.

Traditional static analysis tools (like standard linter rules) suffer from two major flaws:
1. High False Positive Rates: Pure regex-based scanners flag benign code, overwhelming developers with noise.
2. Lack of Contextual & ML Intelligence: Conventional scanners cannot evaluate code intent or semantic context.

Project Objective: This platform was created to bridge this critical gap by unifying **AST Structural Parsing**, **Rule-based SAST Heuristics**, **HuggingFace CodeBERT Deep Learning Embeddings**, and an **AI Security Assistant** into a unified, high-performance web platform.

2. Practical Utility & Real-World Applications

The AI Secure Code Intelligence Platform provides direct utility across multiple domains in the software ecosystem:

Target Audience / Domain	Practical Real-World Utility
Software Developers	Serves as an automated, instant secure code reviewer. Provides auto-generated secure code refactorings so developers learn secure coding practices in real time.
Security Analysts & Auditors	Automates end-to-end repository audits. Generates compliance-ready PDF, HTML, CSV, and JSON audit reports for vulnerability management.
DevSecOps Pipelines	Embeds static security scanning into Continuous Integration / Continuous Deployment (CI/CD) pipelines to block insecure code commits before production deployment.
Computer Science Education	Functions as an interactive learning playground where students inspect Abstract Syntax Tree (AST) structures and learn OWASP Top 10 & CWE mitigation strategies.

3. Technical Deep Dive & System Architecture

The platform implements a multi-layered defensive strategy combining deterministic rules with deep neural networks:

A. Abstract Syntax Tree (AST) Analysis Engine (backend/ast_parser.py)

Unlike basic text-search scanners, the platform parses Python source code into an Abstract Syntax Tree using Python's native ast library. It traverses call nodes to detect dangerous built-in expressions like eval(), exec(), and unsafe pickle.loads() deserializations.

B. Deep Learning Model (CodeBERT Integration) (backend/ml_engine.py)

To evaluate code context, the platform utilizes HuggingFace's **microsoft/codebert-base** model built on PyTorch. CodeBERT processes code tokens and outputs 768-dimensional vector representations ([CLS] embeddings). A classifier trained on top of these embeddings predicts a continuous **ML Vulnerability Risk Score (0-100%)** and confidence interval.

C. Interactive 3D Glassmorphism Frontend (frontend/app.py)

The user interface is crafted using Streamlit enhanced with custom CSS Glassmorphism effects (frosted-glass overlays, deep shadows) and interactive **3D Plotly threat vector scatter plots**, enabling users to analyze vulnerability distribution in 3D space.

4. Step-by-Step Execution Guide (How to Run)

Follow these exact instructions to launch and run the application on your computer:

Step 1: Open Terminal & Navigate to Project Directory

```
cd "C:\Users\Soooraj Mohan\Desktop\ai_code_audit"
```

Step 2: Start the FastAPI Backend Server

```
uvicorn backend.api:app --reload
```

Step 3: Start the Streamlit Frontend Dashboard (in a second terminal)

```
streamlit run frontend/app.py
```

Step 4: Access the Dashboard & Log In

Open your browser to **http://localhost:8501**
• **Username:** admin
• **Password:** admin

5. Academic Project Diary / Logbook Timeline

Use these entries for writing your daily project logbook entries:

Timeline / Phase	Development Activity Log
Day 1 – Day 2 Scope & SAST Setup	Defined project objectives. Built deterministic SAST scanner (backend/scanner.py) targeting SQLi, Command Injection, and hardcoded secrets.
Day 3 – Day 4 AST & Database	Developed AST structural parser (backend/ast_parser.py). Formulated SQLite schema with SQLAlchemy ORM (database/models.py).
Day 5 – Day 6 CodeBERT ML Engine	Integrated HuggingFace microsoft/codebert-base via PyTorch (backend/ml_engine.py) to extract 768-dim embeddings.
Day 7 – Day 8 GitHub & OWASP KB	Created Git repository auditing engine (backend/github_scanner.py) and OWASP Top 10 Knowledge Base (knowledge_base/kb_data.py).
Day 9 – Day 10 3D UI & Authentication	Constructed Streamlit dashboard (frontend/app.py) featuring user authentication, 3D Glassmorphism CSS, and 3D Plotly scatter charts.
Day 11 – Day 12 Reports & Version Control	Implemented ReportLab PDF, CSV, and HTML exporters (backend/report_generator.py). Performed integration testing and Git commits.